

Microcontrollers — Homework 1

Interfacing Techniques

ENCS4380

Ports & Bit-Masking · millis() Multitasking · TimerLib · LCD (Port-Level) · Hardware Interrupts

Course	Platform	Simulation	Total marks
Microcontroller Systems	Arduino Uno (ATmega328P)	TinkerCad / Wokwi / Hardware	100 pts (5 × 20)

General Instructions

- You may simulate in TinkerCad, Wokwi, or Proteus, and run on real hardware. TinkerCad screenshots are accepted in place of a physical demo.
- Wherever a problem forbids a function (e.g. `digitalWrite`, `delay`), using it voids the marks for that specific part — not the whole question.
- Direct AVR I/O means: configure direction with `DDRx`, write with `PORTx`, read with `PINx`. Always use read-modify-write (`|=`, `&=` `~`) to avoid overwriting unrelated bits.
- Submit one `.ino` file per question plus a wiring screenshot/photo and a short report (half a page per question) describing your approach and any measurements.
- Only these Arduino helpers are globally allowed unless a problem says otherwise: `millis()`, `micros()`, `Serial.*`.
- Late submissions lose 10% per day (maximum 3 days).

Tip: All five questions build on each other. The LCD driver you write in Q1 is reused in Q2–Q5. Start early!

Question 1 — Bare-Metal HD44780 LCD Driver (no library, port masking)

In this question you will replace the LiquidCrystal library with your own driver written entirely in direct port I/O. Understanding this level of hardware control is the heart of embedded systems.

Wiring (mandatory)

LCD pin	ATmega328P connection
RS	PB0 (D8) — bit 0 of PORTB
RW	GND (write-only)
EN	PB1 (D9) — bit 1 of PORTB
D4	PD4 (D4) — bit 4 of PORTD
D5	PD5 (D5) — bit 5 of PORTD
D6	PD6 (D6) — bit 6 of PORTD
D7	PD7 (D7) — bit 7 of PORTD
VSS / VDD / V0	GND / +5 V / 10 kΩ contrast pot

Functions to implement

- `void lcd_init(void)` — 4-bit initialisation sequence from the HD44780 datasheet (function set, display ON, entry mode, clear).
- `void lcd_command(uint8_t cmd)` — send a command byte as two 4-bit nibbles; RS = 0.
- `void lcd_data(uint8_t b)` — send a data byte; RS = 1.
- `void lcd_setCursor(uint8_t col, uint8_t row)` — set DDRAM address (row 0 → 0x00, row 1 → 0x40).
- `void lcd_print(const char *s)` — write a null-terminated string character by character.

Demo program

- Row 0: display your full name and student ID (static text).
- Row 1: display a seconds counter that increments every second using `millis()` — no `delay()`.
- The counter must count up correctly and never freeze.

Rules

- Banned entirely: `LiquidCrystal`, `digitalWrite`, `digitalRead`, `pinMode`, `delay()`, `delayMicroseconds()`.
- Use `micros()` for the mandatory HD44780 timing delays ($\geq 40 \mu\text{s}$ after sending data, $\geq 1.64 \text{ ms}$ after clear command).
- Use read-modify-write masking at all times so pins not connected to the LCD are unaffected.

Starter skeleton

```
#define DATA_MASK 0xF0 // PD4..PD7 carry D4..D7
#define RS_BIT (1 << PB0)
#define EN_BIT (1 << PB1)

void lcd_pulse_en() {
    PORTB |= EN_BIT;
```

```

    __asm__ __volatile__("nop\n nop\n nop\n nop\n"); // ≥ 450 ns
    PORTB &= ~EN_BIT;
}

void lcd_send_nibble(uint8_t nib) {
    PORTD = (PORTD & ~DATA_MASK) | (nib & DATA_MASK); // mask keeps other bits safe
    lcd_pulse_en();
}

void lcd_send(uint8_t byte, uint8_t rs) {
    if (rs) PORTB |= RS_BIT; else PORTB &= ~RS_BIT;
    lcd_send_nibble(byte & 0xF0); // high nibble first
    lcd_send_nibble((byte << 4) & 0xF0); // low nibble second
    // wait ≥ 40 µs here using micros()
}

```

Grading rubric (20 pts)

Criterion	Pts	What we look for
DDR and PORT setup with correct masking	4	No direct setting of full port byte; only the required bits changed.
4-bit init sequence works on cold boot	5	Matches HD44780 datasheet; LCD shows content immediately after power-on.
lcd_print and lcd_setCursor work correctly	4	Name on row 0, counter on row 1, no garbled characters.
Counter uses millis() with no delay()	4	Non-blocking main loop; counter increments smoothly.
Report: schematic + timing explanation	3	Explains nibble order and the two timing waits.

Question 2 — Non-Blocking LED Patterns with millis() (no delay)

Build a system where three LEDs blink simultaneously at different rates and a push-button toggles a fourth LED — all without ever calling delay(). Use direct port I/O throughout.

Task specification

- LED A on PB0 (D8): blink every 500 ms (1 Hz).
- LED B on PB1 (D9): blink every 300 ms (~1.67 Hz).
- LED C on PB2 (D10): blink every 750 ms — with a phase offset of 375 ms so it starts ON when LEDs A and B start OFF.

- Button on PD2 (D2) (read via PIND, internal pull-up via PORTD |= (1<<PD2)): each press toggles LED D on PB3 (D11).
- Serial monitor prints a one-line status message every 2 seconds, e.g. "A=1 B=0 C=1 D=0".

millis() pattern reminder

```
uint32_t prevA = 0;
const uint32_t PERIOD_A = 500;

void loop() {
    uint32_t now = millis();

    if (now - prevA >= PERIOD_A) {
        prevA = now;
        PORTB ^= (1 << PB0); // toggle LED A
    }
    // ... same pattern for B and C with their own prevX and PERIOD_X
}
```

Rules

- No delay(), no delayMicroseconds(), no digitalWrite(), no pinMode().
- All LED writes and button reads must go through DDRx / PORTx / PINx registers.
- Add software debouncing for the button (ignore presses within 50 ms of the last valid press).

Bonus (+3 pts)

Add a fifth LED on PB4 that lights up only when LEDs A, B, and C are all ON at the same time. Use a bitmask comparison on PORTB to determine this in a single expression.

Grading rubric (20 pts)

Criterion	Pts	What we look for
All three LEDs blink at correct independent rates	6	Measured visually or with TinkerCad oscilloscope.
Phase offset on LED C	3	C is ON while A/B are OFF at t=0; shown in report.
Button toggles LED D with debouncing	5	No double-triggers; 50 ms threshold referenced in code.
Only register I/O used (no digitalWrite etc.)	4	Code review.
Serial status and report	2	Status message correct; report notes chosen periods.

Question 3 — Periodic Tasks with the Timer Library

Use the Timer library's `every()`, `oscillate()`, and `pulse()` functions to coordinate several timed actions, then display live status on the LCD you built in Q1.

Task specification

- LED 1 on PB0: call `t1.oscillate(LED1_PIN, 400, HIGH)` so it toggles every 400 ms automatically.
- LED 2 on PB1: call `t2.pulse(LED2_PIN, 100, HIGH)` triggered by a timer: fire a 100 ms pulse every 2 seconds using `t3.every(2000, triggerPulse)`.
- Event counter: use `t4.every(1000, countTick)` to increment an integer every second.
- LCD row 0: show "Ticks: XXX" (the counter from above), refreshed every 500 ms using another `every()` call.
- LCD row 1: show "L1:osc L2:pulse" as static status text, updated to "L1:osc L2:off" for 200 ms after each pulse ends (use a `millis()` flag — do NOT add more timers).

Library reminder

```
#include <Timer.h>

Timer t1, t2, t3, t4, t5;

void setup() {
    t1.oscillate(8, 400, HIGH);           // LED on pin 8 toggles every 400 ms
    t3.every(2000, triggerPulse);         // fires triggerPulse() every 2 s
    t5.every(500, updateLCD);              // refresh LCD every 500 ms
}

void loop() {
    t1.update(); t2.update();
    t3.update(); t4.update(); t5.update();
}
```

Rules

- The LCD must use your own driver from Q1 (no `LiquidCrystal` here).
- No `delay()` anywhere in the sketch.
- The pulse LED must not stay on if the Arduino resets mid-pulse; explain how you handle this.

Bonus (+2 pts)

Add a mode where pressing a button (any pin, via PIND register) halves all timer periods for 5 seconds, then restores them — without re-calling `oscillate()`.

Grading rubric (20 pts)

Criterion	Pts	What we look for
oscillate() and pulse() visibly correct	5	LED 1 toggles every 400 ms; LED 2 gives 100 ms pulses every 2 s.
Tick counter correct at 1-second intervals	4	Verified in report over 30 s.
LCD rows update without blocking	5	500 ms refresh via every(); no flicker.
"L2:off" indicator for 200 ms	3	millis() flag used, not an extra timer.
Report: sketch description + observations	3	Timer drift (if any) noted; pulse correctness discussed.

Question 4 — Hardware Interrupts: attachInterrupt() and Register-Level

Explore hardware interrupts at two abstraction levels: the friendly Arduino API, and the raw AVR register interface. You will build a button-driven event counter and compare both approaches.

Part A — Using attachInterrupt() (10 pts)

- Wire Button 1 to INT0 (D2) and Button 2 to INT1 (D3).
- Button 1 (falling edge): increment a counter `count`.
- Button 2 (falling edge): reset `count` to zero.
- Display `count` on the LCD (row 0) and a log line on Serial every time the counter changes.
- Apply software debouncing inside the ISR using a `millis()`-based guard (≥ 50 ms between accepted edges).

```
volatile uint16_t count = 0;
volatile uint32_t lastPressMS = 0;

void onButton1() {
    uint32_t now = millis();
    if (now - lastPressMS >= 50) { // debounce
        count++;
        lastPressMS = now;
    }
}

void setup() {
    attachInterrupt(digitalPinToInterrupt(2), onButton1, FALLING);
    attachInterrupt(digitalPinToInterrupt(3), onButton2, FALLING);
}
```

Part B — Register-level interrupt (10 pts)

- Remove the two `attachInterrupt()` calls and replace them with direct AVR register configuration.
- Set INT0 to trigger on a falling edge: `EICRA |= (1<<ISC01); EICRA &= ~(1<<ISC00);`.
- Set INT1 to trigger on a falling edge: `EICRA |= (1<<ISC11); EICRA &= ~(1<<ISC10);`.
- Enable both interrupts: `EIMSK |= (1<<INT0) | (1<<INT1);`.
- Write the ISRs with `ISR(INT0_vect) { ... }` and `ISR(INT1_vect) { ... }` — same debounce logic as Part A.
- The observable behaviour must be identical to Part A. Document what each EICRA/EIMSK bit does in your report.

Learning goal: *The key learning: `attachInterrupt()` is a wrapper around EICRA/EIMSK. Both approaches produce the same machine code; the register path just makes the hardware visible.*

Rules

- Part B must not contain any call to `attachInterrupt()` or `detachInterrupt()`.
- ISRs must be short — no `Serial.print()` inside an ISR (set a flag, print in `loop()`).
- LCD display must use your driver from Q1 (no `LiquidCrystal`).

Bonus (+2 pts)

Add a third mode (toggled by a third button via PORTD directly, no interrupt) that pauses counting (ISR still fires but count does not change). Show a "PAUSED" message on the LCD.

Grading rubric (20 pts)

Criterion	Pts	What we look for
Part A: <code>attachInterrupt</code> works, debounced	5	Counter increments/resets reliably; no double-count.
Part B: EICRA / EIMSK set correctly	5	Registers explained in report; behaviour identical to Part A.
ISR uses flag, <code>Serial</code> prints in <code>loop()</code>	3	No <code>Serial.print()</code> inside ISR.
LCD shows live count	4	Q1 driver used; no <code>LiquidCrystal</code> .
Report: register-level explanation	3	Bit-level meaning of EICRA/EIMSK described clearly.

Question 5 — Combined Challenge: Traffic-Light Controller with Pedestrian Crossing

Bring together everything from this homework: direct port I/O, millis() multitasking, TimerLib, hardware interrupts, and your LCD driver — in a realistic embedded system.

System description

- Traffic LEDs on PORTB — Red (PB0), Yellow (PB1), Green (PB2). Control with bitmasks only.
- Pedestrian button on INT0 (D2) configured at register level (EICRA/EIMSK — no attachInterrupt). Pressing the button sets a volatile bool pedRequest flag.
- Normal traffic cycle (millis()-driven, no delay()):

Phase	Duration	LEDs & LCD
GREEN	5 s	Green ON → LCD row 0: "GO"
YELLOW	2 s	Yellow ON → LCD row 0: "SLOW"
RED	5 s	Red ON → LCD row 0: "STOP"
PED CROSSING	4 s	Red ON + TimerLib oscillate() blinks pedestrian LED at 2 Hz → LCD: "CROSS"

- Pedestrian request: if pedRequest is true and traffic is in the GREEN phase, finish the remaining GREEN time then jump immediately to PED CROSSING. Ignore presses during RED or PED CROSSING.
- LCD row 1: always shows the time remaining in the current phase as "T: X s", updated every second with a Timer.every(1000, updateLCD) call.
- Use Timer.oscillate() for the pedestrian crossing blink LED (on PB3) — do not implement this blink manually.

State-machine sketch

```
enum State { GREEN, YELLOW, RED, PED_CROSS };
State state = GREEN;
uint32_t stateStart = 0;
volatile bool pedRequest = false;

ISR(INT0_vect) { pedRequest = true; } // short ISR — only set flag

void loop() {
    uint32_t now = millis();
    uint32_t elapsed = now - stateStart;

    switch (state) {
    case GREEN:
```



```

    if (pedRequest && elapsed >= ??) { // transition early on request
        changeState(PED_CROSS);
    } else if (elapsed >= 5000) {
        changeState(YELLOW);
    }
    break;
    // ... YELLOW → RED → PED_CROSS → GREEN
}
t_lcd.update(); // timer for LCD refresh
t_ped.update(); // timer for oscillate (only active in PED_CROSS)
}

```

Rules

- LED control via PORTB bitmasks only — no digitalWrite.
- INT0 via EICRA/EIMSK — no attachInterrupt().
- TimerLib used only for LCD refresh and pedestrian blink — traffic-phase timing via millis().
- LCD from Q1 — no LiquidCrystal.

Bonus (+4 pts)

Add a 7-segment display (or second LCD row) showing a countdown within each phase. Drive it directly from PORTD using bit masking with a custom 7-segment encoding lookup table stored in PROGMEM.

Grading rubric (20 pts)

Criterion	Pts	What we look for
Correct 3-phase traffic cycle with millis()	5	Durations measured; no phase skipped; no delay() used.
Pedestrian interrupt via EICRA/EIMSK	4	ISR sets flag only; registers correctly configured.
Pedestrian crossing triggered mid-GREEN	4	Transition happens at correct point; request ignored in RED/PED.
TimerLib oscillate() for ped blink + LCD refresh	4	Both calls present; blink rate correct (2 Hz).
Q1 LCD driver reused correctly	3	Both LCD rows update; no LiquidCrystal anywhere.

Submission Checklist

- One ZIP named LastName_FirstName_HW2.zip

- Five folders (Q1..Q5) each containing: the .ino file, a wiring diagram or TinkerCad screenshot, and a half-page PDF report
- A single demo video (≤ 5 minutes) covering all five questions — TinkerCad screen recording is accepted
- Your Q1 lcd_driver is reused in Q2–Q5 (submit it once in Q1; just #include or paste it in the others)

TinkerCad users: *TinkerCad does not support the Timer library by default. For Q3 and Q5 you may use Wokwi (which supports most Arduino libraries) or real hardware for those two questions only.*

Grade summary

Q	Topic	Marks
1	Bare-Metal HD44780 LCD Driver (port masking, no library)	20
2	Non-Blocking LED Patterns — millis() multitasking	20
3	Periodic Tasks — Timer library every/oscillate/pulse	20
4	Hardware Interrupts — attachInterrupt() vs register-level	20
5	Combined Challenge — Traffic-Light Controller	20
	Total	100